

Synchronising POSIX Threads

António Barros
June 26, 2026

1 Mutex and Condition Variable Quick Reference

This cheat sheet summarises the essential POSIX functions for synchronisation. These mechanisms are crucial for managing shared resource access and thread coordination.

Function	Description
<code>pthread_mutex_init()</code>	Initialises a mutex with specified attributes.
<code>pthread_mutex_destroy()</code>	Destroys a mutex, releasing its resources.
<code>pthread_mutex_lock()</code>	Locks the mutex; blocks if it is already locked by another thread.
<code>pthread_mutex_unlock()</code>	Unlocks the mutex, allowing other threads to acquire it.
<code>pthread_cond_init()</code>	Initialises a condition variable with specified attributes.
<code>pthread_cond_destroy()</code>	Destroys a condition variable, releasing its resources.
<code>pthread_cond_wait()</code>	Atomically releases the mutex and suspends the thread until the condition is signalled.
<code>pthread_cond_signal()</code>	Unblocks at least one thread waiting on the specified condition variable.
<code>pthread_cond_broadcast()</code>	Unblocks all threads currently waiting on the specified condition variable.

Table 1: Mutex and condition variable synchronisation functions

2 Exercises: Thread Synchronisation with Mutexes

This set of exercises focuses on preventing race conditions and ensuring data consistency in multi-threaded programmes. In each case, ensure that the use of mutexes is minimal and efficient to avoid unnecessary performance degradation.

2.1 Thread-Safe Shared Counter

Implement a program where four threads simultaneously increment a global shared counter. Each thread must perform the increment operation 20,000 times. Ensure that the final value of the counter is exactly 80,000 using a mutex to protect the critical section.

2.2 Bank Account Transactions

Consider a system where an account balance is represented by a global variable:

```
int balance = 1000;
```

Create a multi-threaded application with a *depositor* thread and a *withdrawer* thread:

- The **depositor** must add 100 to the balance 500 times.
- The **withdrawer** must subtract 100 from the balance 500 times.

To simulate a complex operation, include a short delay (e.g., `usleep(100)`) between reading the balance and writing the updated value.

Use a mutex to ensure that updates to the account balance are atomic, preventing inconsistent states during concurrent read-modify-write operations.

At the end, check if the balance is correct.

2.3 Producer-Consumer using Mutex-only Synchronisation (Polling with Back-off)

Implement a multi-threaded application to manage a shared buffer using **exclusively mutexes**. Condition variables must not be used.

Data Encapsulation

Define a structure to encapsulate the buffer state and its associated synchronization primitive:

```
typedef enum {
    DATA_EMPTY,
    DATA_READY
} status_t;

typedef struct {
    int value;
    status_t status;
    pthread_mutex_t lock;
} buffer_t;
```

Producer Logic

- Lock the mutex and check if the status is `DATA_EMPTY`.
- If full: Unlock the mutex, sleep for the defined back-off interval, and retry.
- If empty: Compute the new value (add a random offset between -5 and 5 to the current value), write it to the buffer, set the status to `DATA_READY`, unlock the mutex, and sleep for the defined interval before repeat.

Consumer Logic

- Lock the mutex and check if the status is `DATA_READY`.
- If empty: Unlock the mutex, sleep for the defined back-off interval, and retry.
- If ready: Read the value, set the status to `DATA_EMPTY`, unlock the mutex, and sleep for the defined interval before repeat.

Defining Timing Parameters and Back-off Intervals

The interval and back-off durations should be stated as macros at the top of the source code file, as follows:

```
#define P_INTERVAL 100000 // Producer sleep time (us)
#define P_BACKOFF 40000 // Producer retry delay (us)
#define C_INTERVAL 100000 // Consumer sleep time (us)
#define C_BACKOFF 20000 // Consumer retry delay (us)
```

Experimental Tasks – Changing Interval Times

Evaluate the system behaviour by running the simulation with the following sleep-time pairs (Producer / Consumer):

- **P: 500 ms / C: 200 ms**
- **P: 200 ms / C: 500 ms**
- **P: 100 ms / C: 800 ms**

Experimental Tasks – Changing Number of Threads

Evaluate the system behaviour by running the simulation with 4 producers and 2 consumers, with the following sleep-time pairs (Producer / Consumer):

- **P: 500 ms / C: 100 ms**
- **P: 500 ms / C: 500 ms**

3 Exercises: Condition Variables

This set of exercises focuses on energy-efficient mechanisms for threads to await the conditions required to enter a critical section.

3.1 Producer-Consumer using Condition Variables

Adapt the final exercise from the previous section by replacing the polling mechanism (which utilises a back-off policy) with a synchronisation approach based on condition variables.

Your implementation should ensure that the producer and consumer threads interact correctly; specifically:

- **Condition Variables:** Define two condition variables that represent the states required for threads to progress into their respective critical sections: `buffer_write_available` and `buffer_data_ready`. These two condition variables must be integrated into the `buffer_t` structure alongside the associated mutex.
- **Producer Logic:** The producer must block when the buffer is not available for writing, efficiently waiting until the consumer removes an item.
- **Consumer Logic:** The consumer must block when no data is available, waiting until the producer adds a new item.
- **Signalling:** Ensure that `pthread_cond_signal()` is correctly invoked after modifying the buffer state to notify waiting threads.

Ensure you handle the spurious wake-up phenomenon by wrapping your condition checks within a `while` loop, rather than an `if` statement.

3.2 Bounded-Buffer with Circular Array

A bounded circular buffer is an efficient and widely adopted mechanism for streaming data between collaborating threads.

In this exercise, you will implement a bounded buffer to manage data samples acquired from sensors. Each element within the buffer must be represented by a struct containing a sample value (int) and the corresponding sensor identifier (char[12]):

```
typedef struct {  
    int value;  
    char sensor[12];  
} sample_t;
```

Task Requirements

- **Data Structure:** Define a `buffer_t` structure containing:
 - An array of 10 elements of your custom data type.
 - A counter for the current number of elements filled.
 - Indices for the next position to read (`read_idx`) and write (`write_idx`).
 - A mutex for mutual exclusion.
 - Two condition variables: `buffer_can_read` and `buffer_can_write`.
- **Circular Logic:** Implement the circular buffer logic. When the pointer reaches the end of the array, it must “wrap around” to the beginning.
- **Thread Safety:** Ensure that producers block when the buffer is full and consumers block when the buffer is empty, using the provided condition variables.

Simulation Logic

- **Cyclic Execution:** Both producer and consumer threads must execute infinite loops.
- **Producer Behaviour:** Upon producing a sample and inserting it into the buffer, the producer thread must sleep for a random interval chosen from the set {100, 200, 500, 1000} milliseconds. This simulates the latency associated with physical sensor interruption events.
- **Consumer Behaviour:** Consumer threads must be designed to be constantly available, immediately processing samples as soon as they become available in the buffer.

Implementation Note: Circular Indexing

To implement the circular wrapping of indices, use the modulo operator (%). When incrementing an index after a write or read operation, update it as follows:

```
index = (index + 1) % 10;
```

This approach ensures that when the index reaches 10, the result of the division remainder resets it to 0, effectively creating a circular traversal of the array without requiring conditional if blocks for bounds checking.

Experimental Analysis

Once your implementation is functional, you are required to assess the behaviour and efficiency of your bounded buffer under different concurrency loads. Execute your program with the following configurations and document your observations:

- **Scenario A:** 4 producers and 1 consumer.
- **Scenario B:** 12 producers and 1 consumer.

- **Scenario C:** 12 producers and 2 consumers.

Reflection Points

When analysing your results, consider the following questions:

1. How does the contention on the mutex evolve as the number of producer threads increases?
2. Does the consumer thread experience periods of starvation, or is it able to maintain pace with the producers?
3. How does the frequency of `pthread_cond_signal()` influence overall throughput? Furthermore, analyse whether `pthread_cond_broadcast()` would be more appropriate in either scenario and justify your reasoning.